LLMOps: Experimentation

This is the point where you move from “cool demo” to “are we actually going to ship this.” It’s the Formal Experimentation phase from the LLMOps Tool Stack, and it maps pretty cleanly to the Frontier / Feature Build patterns in the blog:

- LLMOps Tool Stack (Loom): LLMOps Tool Stack
- “Probabilistic Wall” blog: Your Product Roadmap Just Hit a Probabilistic Wall: A Practical Guide to Building on LLMs

The goal here is simple to say and a bit annoying to do well: for one specific Loom use case, you want to decide with clear metrics whether the system is good enough, under what constraints, and what exact configuration you trust.

I’m going to walk through six steps:

Step 1 – Define success and scope
Step 2 – Build a small but real dataset
Step 3 – Design rubrics and graders
Step 4 – Pick a harness / framework
Step 5 – Test and experiment
Step 6 – Wrap up

You can read this as a story the first time. Later, you’ll use each step as a checklist you mentally walk through before you tell anyone “the model looks pretty good.”

Step 1 – Define success and scope

⚠ Start here. If you skip this, everything else becomes hand-wavy and you’ll have long debates with no clear end point.

First, force yourself to pick exactly one concrete use case. Not “AI summarization for Loom” as a whole. Something like “daily standup Looms to Slack summaries,” or “classify Looms by meeting type,” or “extract action items and owners from project update Looms.” If you can’t say the use case in one sentence without taking a breath, it’s probably too broad.

Once you have that, write this sentence and actually fill in the brackets:

> “The system should **[do X]** with **[Y metric] ≥ [threshold]** under **[constraints]**.”

A few Loom-flavored examples:

- “The system should extract action items from Loom transcripts with F1 ≥ 0.80 on the eval set, under p95 latency ≤ 2 seconds and cost ≤ $0.02 per request.”
- “The system should classify Looms into {standup, retro, demo, 1:1, other} with macro-F1 ≥ 0.75, under p95 latency ≤ 3 seconds.”
- “The system should generate customer-ready summaries for support Looms with an average human usefulness score ≥ 4.0 out of 5, under p95 latency ≤ 4 seconds and cost ≤ $0.03 per request.”

If the team cannot agree on that sentence, you are not ready to run serious evals. Go back, tighten scope, or clarify the product goal. This is a classic “probabilistic wall” failure mode: you can’t define success cleanly, so you get stuck.

Now, metrics. You want between one and six core metrics. There are four natural buckets, and you’ll usually pick at least one metric from the first bucket and then sprinkle in others as needed.

1. Task success
   This is “did the thing do the thing.” For classification-type problems, that’s accuracy, precision, recall, and F1. For extraction problems, it might be F1 on fields like “action_items”. For summarization or generation, it might be a 0–5 rubric score from a judge that roughly means “how good is this for the user.”
2. Safety and policy adherence
   This is about bad behavior: leaking sensitive content, generating dangerous or disallowed content, saying wrong things about customers. You might track “policy violation rate” on your eval set, or a “hallucination rate” if you can define hallucinations clearly for your task.
3. Format and schema correctness
   A lot of Loom features emit structured outputs: JSON, lists, references. Here you care that the JSON is valid, required fields are present, and things like Loom URLs or Jira keys are well-formed. You can measure “schema correctness rate,” e.g. “99.5% of responses pass JSON schema validation.”
4. Latency and cost
   Users don’t care about your clever prompt if the summary shows up five seconds after they’ve moved on. You’ll usually at least set a p95 latency target and a cost per request ceiling.

For each metric you choose, be blunt about whether it is a hard gate or just something you monitor. A hard gate is “if we don’t hit this number, we do not ship or roll out.” A monitoring metric is “we’ll watch this and maybe tighten later, but it doesn’t block us right now.”

A typical Loom table in your head might look like this:

Metric	Target / Constraint	Type	Metric bucket

Task F1 for “needs_followup” classification	≥ 0.80	Hard gate	Task success
Policy violation rate	≤ 0.5%	Hard gate	Safety / policy
JSON schema correctness	≥ 99.5%	Hard gate	Format / schema
Tone / style score (0–5)	≥ 3.5 on average	Monitor	Tone / style
p95 latency	≤ 2 seconds	Hard gate	Latency
Cost per request	≤ $0.02	Monitor (start)	Cost / efficiency

If this sounds like overkill, think about the alternative: weeks into building an AI powered feature, someone asks “is it good enough” and you realize nobody ever defined “good enough” in numeric terms.

See 📊LLMOps: Metrics and Values for a primer on how to choose metrics and target values.

Step 1 checklist

- [ ] One **concrete use case**, not a grab-bag.
- [ ] One sentence success spec filled in.
- [ ] 1–6 metrics chosen and understood by the team.
- [ ] Each metric marked as **Hard gate** or **Monitor**.
- [ ] Numeric targets set for all hard gates.

Step 2 – Build a small but real dataset

Now you need evidence. If you don’t have data that looks like the real world, your eval will happily lie to you.

You want between twenty and two hundred real samples for this phase. You can start with twenty. The reason for that lower bound is not mystical. With ten samples, each error swings your metric by ten percentage points. With twenty, each error is a five-point step. That’s roughly the minimum where you can see if one configuration is really better than another instead of just noise.

The more important part is representativeness. It's much better to have twenty carefully chosen Looms that span the space you care about than two hundred all from the same happy path.

For a Loom feature, "representative" usually means you're covering:

- **Different Loom types**
  Standups, demos, retros, 1:1s, support calls, whatever matches your use case.
- **Different lengths**
  A handful of quick clips under a minute, more meaty Looms around five to ten minutes, and a couple of long beasts if your feature is supposed to handle them.
- **Variations in quality**
  Clear, well-structured Looms where the speaker is organized, plus messy ones with tangents, interruptions, bad mics, or multiple speakers.
- **Known failure modes**
  If you already know certain things break your current feature, those have to be in the dataset.

Examples:

Code-review bot

For a code-review bot, I'd want both tiny PRs and scary large ones. I'd want simple bug fixes and major refactors. I'd want examples where the code is actually clean (no issues to find), as well as examples with each bug pattern we claim to care about. If we say "this feature helps you catch security issues," there should be real security issues in the eval set as well as samples with no security issues, so that you can check for both false positives and false negatives.

Summarization agent

For a summarization agent that turns Looms into standup notes or project updates, I'd want short, punchy Looms where the whole thing is one update, and longer rambling Looms where the speaker drifts a bit. If Loom is used in multiple languages for that use case, I'd want that represented too. And if some teams follow a strict template ("yesterday, today, blockers"), I'd want some of those as well, because the model might overfit to that structure. I would want loud Looms and quiet Looms, Looms recorded in noisy environments, Looms recorded by people with heavy accents or speech impediments, etc.

Where does this data come from? Usually three places.

1. Sampled production Looms, pulled in a privacy-sane way using our internal tooling and pipelines.
2. Existing labeled sets from prior experiments or incident reviews.
3. Synthetic Looms, either recorded or constructed transcripts, for edge cases we don't easily see in the wild.

> It will be very useful to start treating these datasets as shared assets, not one-off experiment junk. The LLMOps Tool Stack calls this out under "dataset management." In practice, for Loom, that means having a shared "dataset library" where teams store:
>
> - A set of Loom IDs or transcripts for a particular eval (e.g. "Loom Summarization Eval v1").
> - A simple manifest listing for each entry: Loom ID, length, type (standup, demo, etc.), language, maybe a few tags like "has_action_items" or "customer_facing."
> - An owner and version number.
>
> Then the next team that wants to evaluate something similar does not start from nothing. They can take your dataset, extend it by adding more Looms and metadata, and reuse the same manifest style. That's how you end up with real, evolving "golden sets" instead of fifteen half-remembered CSVs on laptops.

Now, labels. Some experiments need labels; some can get away with judges alone. Classification experiments obviously need labels: for each Loom, what is the correct class. Extraction tasks often need gold structured data, like a list of action items with owners and due dates. Summarization tasks can use either hand-written "gold summaries" or human scores on model outputs, depending on how formal you want to be.

Decide what labels you need up front. Write a short labeling guideline with a few examples of each label. If the decisions are subjective or tricky, have two people label a subset and compare, so you can see if the instructions make sense.

By the end of Step 2, you should be able to say "we have 50 Looms that cover these meeting types and lengths, we know where they came from, we have labels for X, and they live here with a simple manifest." If the dataset only lives in someone's Downloads folder, you're asking for trouble.

Step 2 checklist

- [ ] At least **20 samples** in the eval set.
- [ ] Representative coverage of user types, Loom types, and edge cases.
- [ ] Data sources and privacy constraints documented.
- [ ] Labeling plan and owner defined.
- [ ] Dataset stored in a shared location with a **manifest**.

Step 3 – Design rubrics and graders

> **If you skipped either of the steps above, stop now. Your eval won't produce meaningful results.**

This is where you decide how you will actually score things. There are really three types of graders in play: pure code, LLM-as-judge, and humans. Most serious Loom evals will use all three.

Start by being greedy about deterministic checks. For each failure mode or metric, ask "can I check this with simple code." If the answer is yes, do that.

Things like JSON schema validation belong here. If your feature is supposed to produce JSON with an `actions` list, each action having `owner`, `due_date`, and `description`, you can write a tiny checker that says yes or no. You can also check things like "does this output contain at least one valid Loom URL," "did any tool call in the trace return an error," or "did we generate more than N bullets when we asked for three."

These checks are boring, but they're incredibly reliable and cheap. They also make perfect hard gates: if JSON is invalid for more than 0.5% of cases, you probably don't want to ship.

Then you get to the mushy stuff. Things like "is this summary actually good," "did it miss any important decisions," or "does the tone feel like Loom" are inherently subjective. This is where LLM-as-judge is useful.

A decent LLM judge setup has three pieces. First, a clear scoring scale. I like 0–2 for simple pass/fail-ish things, or 0–5 if you want more nuance. Second, a rubric that describes each score. For example, for summary quality:

0 means "wrong or unusable, off topic, or major hallucinations."

1 means "partially useful, captures some points, but misses at least one major point or includes minor hallucinations."

2 means "good, captures all key points, no major hallucinations, readable."

Third, examples. Put two or three examples of outputs that deserve each score into the judge prompt, so the judge has concrete anchors. Explicitly tell the judge what to care about and what to ignore. For summarization, that might be "penalize missing key decisions or adding invented ones, ignore minor phrasing differences."

You also need to be aware that judges have biases. If you compare two candidate outputs A and B by telling the judge "here is A, here is B, which is better," the judge's answer can depend on which one you

show first. So if you are running A/Bs, randomize the order and do not always put your favorite first. For really important metrics, you might run the judge multiple times or with slight prompt variations to see if the answer is stable.

Finally, there are cases where you still want humans in the loop. Anything high-stakes, like content that might have legal or security implications, or brand new task types where you do not yet trust an LLM judge, should get a human rater loop.

The key is to not reinvent your rubric there. Use the same rubric you use for LLM judges. Build a small audit set, maybe fifty to a hundred items, where humans have scored each case. Also run your LLM judge on the same set. Then you can look at how often the judge agrees with humans overall and in different slices, like by Loom type, language, or length. That's how you notice "our judge is really bad at long Looms in language X" before it bites you.

Once you've sorted out what's code, what's LLM, and what's human, you're in a much better place to run repeatable experiments rather than ad-hoc eyeballing.

Step 3 checklist

- [ ] Deterministic checks written for anything code can verify.
- [ ] Rubric + LLM-as-judge prompt drafted for subjective metrics.
- [ ] Human rater loop designed for high-stakes cases.
- [ ] Plan for judge drift / agreement checks defined (audit set).

Step 4 – Pick a harness / framework

At this point, you know what success looks like and you have data and graders. Now you need a way to actually run the experiments and collect results.

In Loom's world today, the primary options include Braintrust, LoomEval, and AIX. Here's a rubric you can you in choosing the right tool for the job:

- **Command line or UI**
  If your evals will be mostly run and iterated on by engineers, a CLI and code-first harness like LoomEval or something built on AIX is often preferred. You can check configs into git, call the eval from scripts, and wire it into CI. If PMs, designers, or analysts are going to actively poke at experiments, having a UI with dashboards and run history, like Braintrust, may work better. In practice, teams often end up with a hybrid: code-first harnesses under the hood, plus some UI for inspecting results.

- **Local or hosted**
  Hosted tools host datasets, prompts, and runs on a platform (e.g. Braintrust). That's nice for sharing and for longer-term history but it can mean a little more ceremony and a lot more clicks, and sometimes trickier to debug strange cases or wire in custom code. Local tools live near your code, usually in your repo. That's easy to version, you can tweak harness logic freely, but you need discipline or every engineer ends up with their own private eval script.
- **Lab or in-situ**
  This one is about how close to production you want your eval calls to be. Lab-style evals are the classic "input → model → output" setup run in the eval tool, where you know the entire prompt ahead of time and the harness just calls the model directly. In-situ (black box) evals call the actual running system: a backend that builds prompts dynamically, uses tools, or chains multiple models. For agentic Loom features that decide how to respond based on state, in-situ is often the only way to get realistic behavior.

> In practice, serious features trend toward in-situ evals, because a lot of the interesting logic lives in the orchestration and tool calls, not just the single prompt. That usually means using something like LoomEval or Braintrust wired up to the real service, with AI Gateway in the middle, and having the eval harness hit that service exactly like the product would.

> ⚠ Loom hasn't decided whether or not to continue with Braintrust in FY'27. Use at your own risk.

The exact tool choice will shift over time as the stack evolves, but if you answer those three questions honestly for your team, you'll end up with something workable instead of endlessly debating platforms.

Final note: obsess over the process, not the tool. If you have good data and clear metrics, there are many ways to solve the problem. On the other hand, crappy data and the wrong metrics can't be solved by any tool. In the end, just pick (or vibe) a harness that can do five basic things: read inputs from your dataset, call your model or live feature endpoint, capture outputs plus latency and cost, run your graders, and store results in a way that lets you compare runs.

Step 4 checklist

- [ ] We know if the eval is **lab-style**, **in situ**, or both.
- [ ] We picked a harness our team can run (LoomEval / Braintrust / AIX).
- [ ] We know where datasets and prompts live (fs / git / hosted).
- [ ] Our harness can run our graders and persist results for comparison.

Step 5 – Test and experiment

Now you actually run everything.

Start by defining a **baseline**. That could be the current production configuration if the feature exists, or a very simple initial prompt and model if you're building something new. Either way, write down what the baseline is: which model, which prompt, any special settings. Then run your eval suite on this baseline configuration. This will give you a sense of where you're starting and will be your score to beat.

After that, iterate, but try to be more scientist than gambler. A healthy pattern is to change one major thing at a time. For instance, one experiment could change the prompt wording while keeping the model and everything else fixed. Another might keep the prompt and switch models. Another might tweak retrieval parameters or temperature. After each change, re-run the eval on the same dataset and compare to the baseline and the current best run.

If you change prompt, model, retrieval, and judge all at once, and your metrics move by ten points, you have no idea which change helped and which hurt. That's the chaos version of experimentation. It's fast in the moment and slow overall.

Every time you find a configuration that looks like a serious candidate, take a snapshot. This is more boring bookkeeping but it is what keeps you from losing track. A good snapshot includes the version of the dataset or manifest you used, the exact prompt or prompts, the model name and configuration (temperature, max tokens, any special toggles), the judge setup and rubric version, and the full set of metric results, plus a run ID or URL if your harness provides that.

> Pro Tip: name these snapshots in a way that other people can reference, like "loom-summarization-v3-2026-03-01." Then, when someone later asks "what did we ship," you can point at a specific snapshot rather than vaguely saying "the config from early March."

You keep iterating until one of two things happens. Either you find a configuration that hits or beats your hard gate metrics and has acceptable tradeoffs on latency and cost. Or you try a reasonable number of variants and you're still far away from the targets. In the latter case, the honest move is to say "with current models and this data, this idea is not feasible at our quality bar," and either scope down the problem or park it.

If you like, create a small Confluence table for experiments:

Variant ID	Change description	Primary metric	Δ vs baseline	Notes
v1	Baseline			

v2	Prompt variant A			
v3	Model change			

Step 5 checklist

- [ ] Baseline eval run and metrics recorded.
- [ ] Controlled experiments performed (one major change at a time).
- [ ] A configuration meets or exceeds hard gate metrics, or we documented that it is infeasible.
- [ ] Best run has a **complete snapshot** (dataset, prompt, model, judges, metrics, run ID).

Step 6 – Wrap up

This step sounds like ceremony, but it's the thing that makes your work reusable.

If you're done experimenting, even if the outcome is "we're not shipping this," create a short written summary. It does not need to be a novel. A good summary says what problem you tried to solve, what the one-sentence success spec was, what dataset you used, what metrics you achieved, which configuration ended up best, and what decision you made. Link to your dataset manifest, your snapshots, and any relevant runs in Braintrust, LoomEval logs, or AIX notebooks. (There's a generic form on the templates page.)

Alongside that, treat your dataset like a product. Write a simple data card or manifest that explains what the dataset is for, how you collected it, what Loom types and lengths it covers, what its known gaps are, and who owns it. Store it somewhere that other Loom teams can find, ideally linked from whatever central page you use for evaluators and datasets.

If you are promoting a configuration into production, make that explicit. "This snapshot is our candidate production config." Include its name, prompt version, model version, and any judge differences between lab and production. It's usually worth doing a quick peer review at this stage: walk another PM, EM, or ML partner through the experiment, the metrics, and the decision. They'll often spot missing edge cases or subtle misalignments between the eval and the real user problem.

When you've done all of that, you've basically closed the loop on the Experimentation and Evaluation phase for that use case. You know what "good enough" means numerically, you have data and tools to re-check it later, and you either have a candidate config ready for the production pipeline, or a clear, defensible reason you're not moving forward.

Step 6 checklist

- [ ] Experiment summary page created or updated.
- [ ] Dataset has a **manifest / data card** in a shared location.
- [ ] Candidate production snapshot identified (if applicable).
- [ ] Peer review of the experiment done (PM/EM/AI partner).